SCP-03 Implementation for SE-QUBIP

Hardware Architecture and I2C Integration Guide

Pablo Navarro Torrero
(navarro@imse-cnm.csic.es)

Version: 1.0
December 5, 2025

Contents

1 Introduction

This document details the custom hardware implementation of the **Secure Channel Protocol 03 (SCP-03)** for the SE-QUBIP project. The implementation targets a Xilinx Genesys II platform where a Raspberry Pi 4 Host communicates with a Secure Element (SE) implemented in Programmable Logic.

The SCP-03 module acts as a **Security Proxy** layer. It sits transparently between the external communication interface (I2C) and the internal SE. It intercepts encrypted traffic, authenticates and decrypts it, and passes cleartext data to the internal registers of the SE. Conversely, it intercepts data read from the SE, encrypts it, and signs it before sending it back to the Host.

2 Standards Compliance

This implementation adheres to the GlobalPlatform specifications with specific architectural optimizations for the target hardware environment.

2.1 GlobalPlatform Card Specification v2.2 – Amendment D

We follow the **Secure Channel Protocol '03'** as defined in Amendment D [1].

- **Section 4.1.1 (AES):** We use AES-128 (configurable to 192/256) as the core cryptographic primitive.
- **Section 4.1.5 (Data Derivation):** We implement the KDF in Counter Mode as specified in NIST SP 800-108 [2].
- **Section 6.2 (Cryptographic Usage):**
  - **Message Integrity:** We use CMAC (NIST SP 800-38B) [3] for C-MAC and R-MAC generation and verification.
  - **Confidentiality:** We use AES-CBC (NIST SP 800-38A) [4] for C-ENC and R-ENC.
- **Section 7.1 (Commands):** We implement `INITIALIZE UPDATE` and `EXTERNAL AUTHENTICATE` for session initiation.

2.2 Specific Optimizations and Limitations

The implementation is tailored for a constrained bandwidth environment, specifically a Raspberry Pi 4 acting as the Host Controller communicating via I2C with a Genesys II FPGA. To maximize effective throughput and minimize logic footprint, the following architectural optimizations deviate from a generic GlobalPlatform implementation while maintaining cryptographic compliance:

- **Implicit Protocol Framing:** Standard GlobalPlatform APDUs impose significant byte overhead (CLA, INS, P1, P2, Lc) relative to the small payload sizes typical of register access. To optimize I2C utilization:
  - For **Secure Write** (`CMD_UNWRAP`): The APDU header for the `STORE DATA` (0xE2) command is generated internally by the FPGA logic. The Host transmits only the encrypted payload and the C-MAC, significantly reducing transmission time.
  - For **Secure Read** (`CMD_WRAP`): The FPGA automatically constructs the response structure, injecting the required Status Word (0x9000) and padding prior to R-MAC generation. This allows the Host to request secure data using a minimal 1-byte address pointer rather than a full `GET DATA` APDU.

- **Aligned Burst Granularity:** Data transfers are strictly aligned to a 64-bit granularity, matching the I2C register width of the FPGA interface. This alignment simplifies the internal bridging to the 128-bit AES engine, eliminating the need for complex byte-alignment logic or barrel shifters, thereby reducing critical path delay in the Programmable Logic.
- **Static Key Provisioning:** To minimize the attack surface, Static Root Keys ($K_{ENC}, K_{MAC}, K_{DEC}$) are provisioned via synthesis parameters. The logic focuses exclusively on the Secure Transport of application data rather than Key Management.
- **Enforced Full Security (Level 0x33):** The implementation removes the complexity of dynamic security level negotiation. Once the session is established, the system enforces **Security Level 0x33** (Command Encryption, Command MAC, Response Encryption, Response MAC). Any transaction attempting to downgrade security (e.g., requesting data without R-MAC) is automatically rejected by the FSM, ensuring end-to-end protection for all bidirectional traffic.

3 System Architecture

The proposed architecture establishes a symmetric Secure Channel Protocol (SCP03) between an untrusted Host (Raspberry Pi 4) and a hardware-accelerated Root of Trust on the Genesys II FPGA. The system functions as a cryptographic proxy: the `scp03` hardware module intercepts all traffic on the I2C bus, manages the session state, and transparently forwards only authenticated and decrypted commands to the internal Secure Element (SE).

As illustrated in Figure 1, the architecture segregates the cryptographic workload into two domains. The Host handles the protocol in software, while the FPGA utilizes dedicated hardware accelerators to enforce security boundaries.

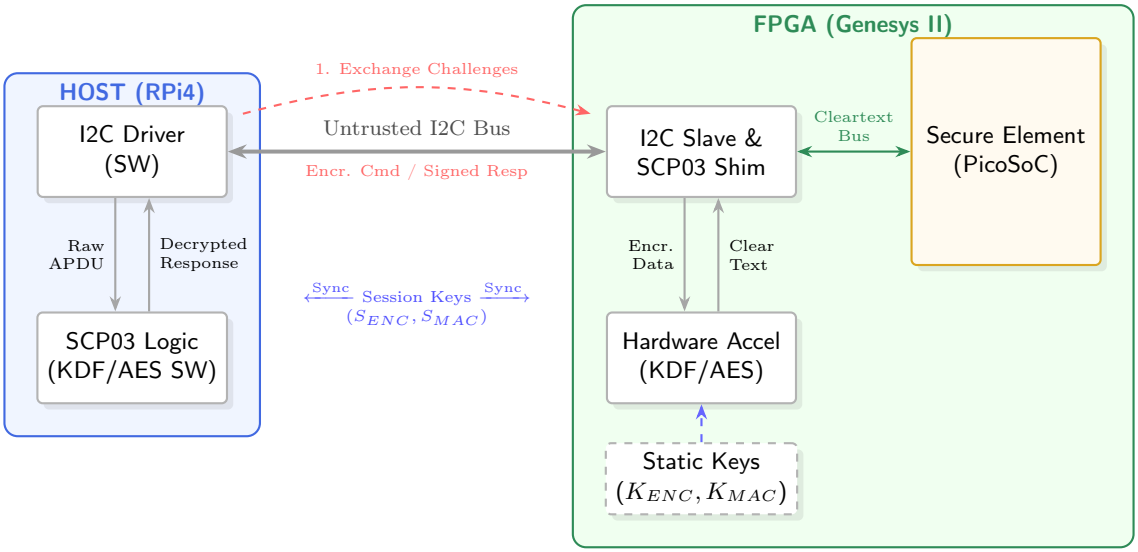

Figure 1: Symmetric Cryptographic Architecture: The FPGA acts as a proxy, decrypting incoming traffic before it reaches the internal Secure Element.

The operation is divided into two distinct phases:

1. **Session Handshake:** The session is established via a mutual authentication process.
   - The Host initiates the sequence by sending a `CMD_INIT_UPDATE` containing a random **Host Challenge**.

- The FPGA responds with a **Card Challenge** generated by an on-chip TRNG. Both entities effectively exchange nonces to prevent replay attacks.
- Using the exchanged challenges and their respective copies of the Static Keys (K_{ENC}, K_{MAC}), both sides execute the NIST SP 800-108 Key Derivation Function (KDF). This results in an identical, ephemeral session key set ($S_{ENC}, S_{MAC}, S_{RMAC}$).
- Mutual authentication is finalized when the Host sends a computed cryptogram and C-MAC, which the FPGA's `SCP03_AES` engine verifies.

2. **Secure Operation (Bidirectional):** Once the session is active, the data transport layer is secured.
   - **Command Path (Host → SE):** The Host encrypts the APDU payload using S_{ENC} and signs it with S_{MAC}. The FPGA `SCP03_AES` module validates the signature and decrypts the payload before forwarding the cleartext command to the internal Secure Element (PicoSoC).
   - **Response Path (SE → Host):** When the SE produces a response, the FPGA Shim intercepts the cleartext data. The hardware accelerator encrypts the response using S_{ENC} and generates a response signature (R-MAC) using S_{RMAC}. The Host driver then receives this package, verifies the R-MAC, and decrypts the data for the application layer.

3.1 FPGA Module Breakdown

The system architecture delegates responsibilities across several specialized sub-modules:

1. **I2C:** Handles the I2C physical layer, synchronizers, and glitch filters. It maps the I2C address space (0x00-0x11) to the SCP03 control signals. It acts as the Master of the internal local bus.
2. **SCP03:** The main FSM. It manages the session state (Idle, Authenticated), buffers incoming data packets, and arbitrates access to the crypto resources.
3. **SCP03_KDF:** A dedicated state machine that implements the NIST SP 800-108 Counter Mode KDF. It drives the AES engine to derive Session Keys ($S_{ENC}, S_{MAC}, S_{RMAC}$) from the Static Keys and Challenges.
4. **SCP03_AES:** The cryptographic engine. It wraps a core AES implementation and adds logic for CBC (Cipher Block Chaining) and CMAC (Cipher-based MAC), including subkey ($K1, K2$) caching and automatic regeneration upon context switching.

4 I2C Register Map and Protocol

Communication is performed over I2C using 64-bit (8-byte) burst transfers. The register map is defined to facilitate packet assembly.

Register	R/W	Name	Description
0x00 – 0x07	W	`DATA_IN`	64-bit Input Buffer. Host writes payload data here.
0x08 – 0x0F	R	`DATA_OUT`	64-bit Output Buffer. Host reads response data here.
0x10	W	`COMMAND`	Control Register to trigger SCP-03 operations.
0x11	R	`STATUS`	Status Register (Busy, Data Available, Authenticated).

Table 1: I2C Register Map

4.1 Control Commands (Register 0x10)

The Host orchestrates the session by writing the following opcodes to Register 0x10.

- **0x10 (`CMD_LOAD_BUF`):** Loads the 64 bits currently in `DATA_IN` into the FPGA's internal assembly buffer. Increments the internal write pointer.
- **0x01 (`CMD_INIT_UPDATE`):** Starts the session handshake. Uses the data in `DATA_IN` as the Host Challenge.
- **0x02 (`CMD_EXT_AUTH`):** Completes authentication. Uses the data in the internal buffer (Host Cryptogram + MAC) to verify the Host.
- **0x11 (`CMD_UNWRAP`):** Secure Write. Decrypts and Verifies the data in the internal buffer and forwards it to the SE.
- **0x12 (`CMD_WRAP`):** Secure Read. Reads data from the SE address specified in `DATA_IN`, encrypts/signs it, and places it in the Output Buffer.

5 Detailed Transaction Flows

The Secure Channel Protocol is implemented through a sequence of state-dependent transactions. Each transaction involves specific interactions with the FPGA's I2C register map to maintain synchronization between the Host's software state and the FPGA's hardware state.

5.1 Phase 1: Session Handshake (Initialize Update)

This initial phase establishes the cryptographic context. It utilizes a mutual exchange of nonces to prevent replay attacks and provides the entropy required to derive unique Session Keys ($S_{ENC}, S_{MAC}, S_{RMAC}$).

1. **Host Challenge Injection:** The Host writes an 8-byte random challenge to the `DATA_IN` register.
2. **Trigger Generation:** The Host issues the `CMD_INIT_UPDATE` (0x01) command.
3. **Hardware Key Derivation:**
   - The FPGA retrieves the Host Challenge.
   - The internal TRNG generates a random 8-byte Card Challenge.
   - The hardware KDF engine combines both challenges with the Static Keys to derive the session keys.
4. **Response Retrieval:** The Host polls the status register. Upon completion, it performs two reads from `DATA_OUT` to retrieve the `Card Challenge` and the `Card Cryptogram`.

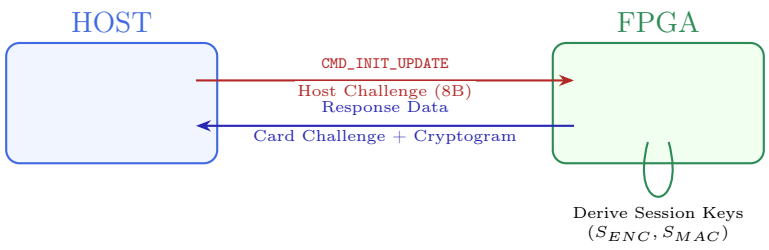

Figure 2: Transaction Flow: Initialize Update

5.2 Phase 2: Mutual Authentication (External Authenticate)

Following the handshake, the Host must prove its identity by demonstrating possession of the Static Keys.

1. **Offline Calculation:** The Host calculates the expected `Host Cryptogram` and a `C-MAC` using the newly derived session keys.
2. **Payload Loading:** The Host uploads the authentication data into the FPGA's internal buffer via two sequential `CMD_LOAD_BUF` transactions (one for the Cryptogram, one for the MAC).
3. **Verification:** The Host issues `CMD_EXT_AUTH` (0x02).
4. **Session Activation:** The FPGA's AES engine verifies the MAC and Cryptogram. If valid, the internal `is_authenticated` flag is set high, unlocking the Secure Channel for data transport.

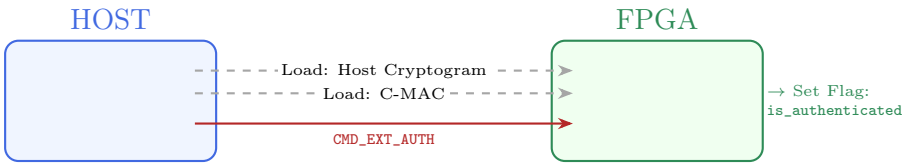

Figure 3: Transaction Flow: External Authenticate

5.3 Phase 3: Secure Write (CMD_UNWRAP)

This command facilitates the confidential delivery of data to the Secure Element. Due to the 64-bit I2C burst constraint, the Host segments the encrypted payload into chunks.

Packet Construction & Segmentation: The Host prepares a 24-byte payload consisting of the Header, Padding, Encrypted Data, and C-MAC. This is split into three 64-bit chunks.

1. **Burst Transfer:** The Host sequentially writes Chunk 1, Chunk 2, and Chunk 3 to `DATA_IN`, triggering `CMD_LOAD_BUF` after each write.
2. **Unwrap Command:** The Host issues `CMD_UNWRAP` (0x11).
3. **FPGA Processing:** The SCP03 module verifies the C-MAC using S_{MAC} and decrypts the payload using S_{ENC}.
4. **Delivery:** If integrity checks pass, the cleartext data is written to the mapped SE register.

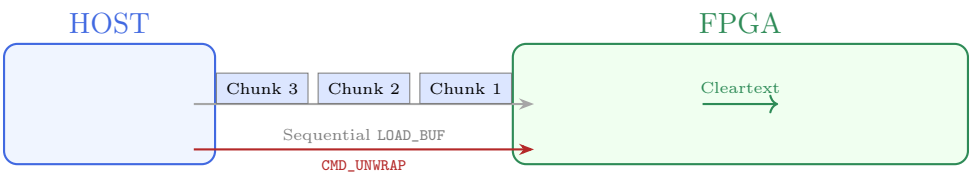

Figure 4: Transaction Flow: Secure Write (Data Unwrap)

5.4 Phase 4: Secure Read (CMD_WRAP)

This operation allows the Host to retrieve data from the SE with integrity and confidentiality protection.

1. **Address Request:** The Host writes the target 1-byte address to the MSB of `DATA_IN` and issues `CMD_WRAP` (0x12).
2. **FPGA Fetch & Wrap:**
   - The FPGA reads the cleartext data from the SE.
   - It automatically applies padding, encrypts the result (S_{ENC}), and appends an R-MAC signature (S_{RMAC}).
3. **Burst Retrieval:** The FPGA places the resulting 24-byte secure packet into the output buffer.
4. **Host Readback:** The Host performs three sequential reads from `DATA_OUT` to retrieve the secure chunks, then locally decrypts and verifies them.

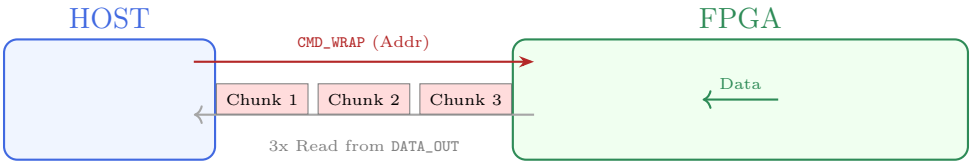

Figure 5: Transaction Flow: Secure Read (Data Wrap)

6 Cryptographic Implementation Details

6.1 AES-CMAC Engine

The core `scp03_aes` module implements a unified AES engine that handles both CBC and CMAC modes.

- **Subkey Caching:** To optimize performance, the engine caches the CMAC subkeys ($K1, K2$) for each key context (S_{MAC}, S_{RMAC}).
- **Key Update Signal:** When the Top Level FSM updates a session key (e.g., after KDF), it asserts `key_update`, forcing the engine to regenerate $K1/K2$ on the next use.

6.2 ICV and Counter Management

Per SCP-03 requirements, the implementation maintains a strictly synchronized 16-byte **Session Counter**.

- **Write (Unwrap):** Uses Counter N for ICV generation. Increments Counter to $N + 1$ after successful decryption.
- **Read (Wrap):** Uses Counter N (with MSB set to `0x80`) for Response IV. Increments Counter to $N + 1$ after successful MAC generation.

This "Always Increment" strategy ensures unique Initialization Vectors (IV) for every single encryption operation, providing robustness against replay and cryptanalysis.

6.3 Random Number Generation and Entropy Sources

The security of the SCP03 mutual authentication phase relies heavily on the uniqueness of the session challenges (Nonces). The architecture employs different strategies for the Host and the FPGA, commensurate with their roles in the security hierarchy.

6.3.1 Host Challenge Generation (PRNG)

The Host software driver implements the **xoshiro256++** algorithm [5] to generate the 8-byte `Host Challenge`. This Pseudo-Random Number Generator (PRNG) was selected for its high performance and statistical robustness.

The initialization process ensures non-determinism across sessions:

- **Entropy Source:** The generator harvests an initial 64-bit seed from the system's current time and the Process ID (PID).
- **State Expansion:** To prevent statistical artifacts during the initial generation phase, the 64-bit seed is expanded into the required 256-bit internal state using the **SplitMix64** algorithm. This ensures the internal state is well-distributed before the first challenge is produced.

6.3.2 Card Challenge Generation (Hardware TRNG)

In contrast to the Host, the FPGA acts as the Root of Trust and therefore implements a **True Random Number Generator (TRNG)**.

The `Card Challenge` is derived from a physical entropy source based on **Ring Oscillators (RO)**. The logic instantiates a chain of inverters in a feedback loop that oscillates at a high frequency. Due to thermal noise and process variations, the phase of these oscillators accumulates jitter over time. The hardware samples this jitter to produce non-deterministic, high-entropy random bitstreams, ensuring that the generated `Card Challenge` is physically unpredictable and unique for every session.

7 Host Integration and Configuration

The Host software driver is designed as a **Transparent Abstraction Layer**. It integrates into the existing Interface API (`INTF`), allowing legacy applications to utilize the Secure Channel without code modifications.

The security context is determined at compile time via the `I2C_SCP03` preprocessor directive. When defined, the standard interface functions automatically wrap the secure session management, ensuring that cryptographic complexity remains hidden from the upper application layer.

7.1 Abstraction Layer Implementation

As shown in Listing 1, the `open_INTF`, `read_INTF`, and `write_INTF` functions act as wrappers.

- **Initialization:** When `open_INTF` is called, the driver automatically performs the `INITIALIZE UPDATE` and `EXTERNAL AUTHENTICATE` handshake via `scp03_init`.
- **Data Transfer:** Standard read/write requests are intercepted and data is routed through `scp03_read`/`scp03_write` for encryption and signing.

```
1 void open_INTF(INTF* interface, size_t address, size_t length)
2 {
3 #ifdef I2C
4     open_I2C(interface);
```

```
    set_address_I2C(*interface, address);
#elif I2C_SCP03
    // Transparently establish Secure Session during interface open
    open_I2C(interface);
    set_address_I2C(*interface, address);
    scp03_init(*interface, &scp03_session);
#else
    createMMIOWindow(interface, address, length);
#endif
}

void close_INTF(INTF interface)
{
#ifdef I2C
    close_I2C(interface);
#elif I2C_SCP03
    close_I2C(interface);
#else
    closeMMIOWindow(&interface);
#endif
}

//-------------------------------------------------------------------
//-- Transparent Read & Write Wrappers
//-------------------------------------------------------------------

void read_INTF(INTF interface, void* data, size_t offset, size_t size_data)
{
#ifdef I2C
    read_I2C_ull(interface, data, offset, size_data);
#elif I2C_SCP03
    // Automatic Secure Read (Wrap)
    scp03_read(interface, &scp03_session, offset >> 3, data);
#else
    readMMIO(&interface, data, offset, size_data);
#endif
}

void write_INTF(INTF interface, void* data, size_t offset, size_t size_data)
{
#ifdef I2C
    write_I2C_ull(interface, data, offset, size_data);
#elif I2C_SCP03
    // Automatic Secure Write (Unwrap)
    scp03_write(interface, &scp03_session, offset >> 3, data);
#else
    writeMMIO(&interface, data, offset, size_data);
#endif
}
```

Listing 1: Transparent Host Driver Implementation (INTF Wrapper)

7.2 Configuration Parameters

To configure the secure channel, the user is only required to modify the `scp03.h` header file. This file defines the AES key length and the provisioned Static Root Keys (K_{ENC}, K_{MAC}), which must match the hardware synthesis parameters.

```
// SCP-03 Configuration
#define SCP03_KEY_BITS          128

// Static Encryption Key (K_ENC)
#define SCP03_STATIC_KEY_ENC    { \
```

```
                                    0x00, 0x01, 0x02, 0x03, 0x04, 0x05, 0x06, 0x07,
    \
                                    0x08, 0x09, 0x0A, 0x0B, 0x0C, 0x0D, 0x0E, 0x0F,
    \
                                    0x10, 0x11, 0x12, 0x13, 0x14, 0x15, 0x16, 0x17,
    \
                                    0x18, 0x19, 0x1A, 0x1B, 0x1C, 0x1D, 0x1E, 0x1F
    \
                                    }

// Static MAC Key (K_MAC)
#define SCP03_STATIC_KEY_MAC    { \
                                    0x20, 0x21, 0x22, 0x23, 0x24, 0x25, 0x26, 0x27,
    \
                                    0x28, 0x29, 0x2A, 0x2B, 0x2C, 0x2D, 0x2E, 0x2F,
     \
                                    0x30, 0x31, 0x32, 0x33, 0x34, 0x35, 0x36, 0x37,
    \
                                    0x38, 0x39, 0x3A, 0x3B, 0x3C, 0x3D, 0x3E, 0x3F
    \
                                    }
```

Listing 2: Static Key Configuration (scp03.h)

References

[1] GlobalPlatform, *GlobalPlatform Card Specification Amendment D: Secure Channel Protocol '03'*, 2014. Document Reference: GPC_SPE_014.

[2] L. Chen, "Nist special publication 800-108: Recommendation for key derivation using pseudo-random functions," special publication (sp), National Institute of Standards and Technology (NIST), October 2009.

[3] M. Dworkin, "Nist special publication 800-38b: Recommendation for block cipher modes of operation: The cmac mode for authentication," special publication (sp), National Institute of Standards and Technology (NIST), May 2005.

[4] M. Dworkin, "Nist special publication 800-38a: Recommendation for block cipher modes of operation: Methods and techniques," special publication (sp), National Institute of Standards and Technology (NIST), December 2001.

[5] D. Blackman and S. Vigna, "Scrambled linear pseudorandom number generators," *ACM Transactions on Mathematical Software*, vol. 47, no. 4, pp. 1–32, 2021.